

References

- [1] S. B. Seidman and B. L. Foster, "A graph-theoretic generalization of the clique concept," *Journal of Mathematical sociology*, vol. 6, no. 1, pp. 139–154, 1978.
- [2] J. Pattillo, N. Youssef, and S. Butenko, "On clique relaxation models in network analysis," *European Journal of Operational Research*, vol. 226, no. 1, pp. 9–18, 2013.
- [3] A. Conte, T. D. Matteis, D. D. Sensi, R. Grossi, A. Marino, and L. Versari, "D2K: scalable community detection in massive networks via small-diameter k -plexes," in *KDD*. ACM, 2018, pp. 1272–1281.
- [4] J. M. Lewis and M. Yannakakis, "The node-deletion problem for hereditary properties is np-complete," *J. Comput. Syst. Sci.*, vol. 20, no. 2, pp. 219–230, 1980.
- [5] B. Balasundaram, S. Butenko, and I. V. Hicks, "Clique relaxations in social network analysis: The maximum k -plex problem," *Oper. Res.*, vol. 59, no. 1, pp. 133–142, 2011.
- [6] D. Berlowitz, S. Cohen, and B. Kimelfeld, "Efficient enumeration of maximal k -plexes," in *SIGMOD*. ACM, 2015, pp. 431–444.
- [7] Y. Zhou, J. Xu, Z. Guo, M. Xiao, and Y. Jin, "Enumerating maximal k -plexes with worst-case time guarantee," in *AAAI*. AAAI Press, 2020, pp. 2442–2449.
- [8] Q. Dai, R. Li, H. Qin, M. Liao, and G. Wang, "Scaling up maximal k -plex enumeration," in *CIKM*. ACM, 2022, pp. 345–354.
- [9] Z. Wang, Y. Zhou, M. Xiao, and B. Khossainov, "Listing maximal k -plexes in large real-world graphs," in *WWW*. ACM, 2022, pp. 1517–1527.
- [10] M. Xiao, W. Lin, Y. Dai, and Y. Zeng, "A fast algorithm to compute maximum k -plexes in social network analysis," in *AAAI*. AAAI Press, 2017, pp. 919–925.
- [11] J. Gao, J. Chen, M. Yin, R. Chen, and Y. Wang, "An exact algorithm for maximum k -plexes in massive graphs," in *IJCAL*. ijcai.org, 2018, pp. 1449–1455.
- [12] H. Jiang, D. Zhu, Z. Xie, S. Yao, and Z. Fu, "A new upper bound based on vertex partitioning for the maximum k -plex problem," in *IJCAL*. ijcai.org, 2021, pp. 1689–1696.
- [13] Y. Zhou, S. Hu, M. Xiao, and Z. Fu, "Improving maximum k -plex solver via second-order reduction and graph color bounding," in *AAAI*. AAAI Press, 2021, pp. 12 453–12 460.
- [14] L. Chang, M. Xu, and D. Strash, "Efficient maximum k -plex computation over large sparse graphs," *Proc. VLDB Endow.*, vol. 16, no. 2, pp. 127–139, 2022. [Online]. Available: <https://www.vldb.org/pvldb/vol16/p127-chang.pdf>
- [15] Z. Wang, Y. Zhou, C. Luo, and M. Xiao, "A fast maximum k -plex algorithm parameterized by the degeneracy gap," in *IJCAL*. ijcai.org, 2023, pp. 5648–5656.
- [16] H. Jiang, F. Xu, Z. Zheng, B. Wang, and W. Zhou, "A refined upper bound and inprocessing for the maximum k -plex problem," in *IJCAL*. ijcai.org, 2023, pp. 5613–5621.
- [17] J. Zheng, M. Jin, and K. He, "Two new upper bounds for the maximum k -plex problem," 2023.
- [18] G. Guo, D. Yan, L. Yuan, J. Khalil, C. Long, Z. Jiang, and Y. Zhou, "Maximal directed quasi-clique mining," in *ICDE*. IEEE, 2022, pp. 1900–1913.
- [19] "Online appendices," 2025, <https://github.com/akhlaqueak/k1k2-plex/blob/main/appendix.pdf>.
- [20] "arabic-2005," <https://sparse.tamu.edu/LAW/arabic-2005>.
- [21] "uk-2005," <https://sparse.tamu.edu/LAW/uk-2005>.
- [22] "it-2004," <https://sparse.tamu.edu/LAW/it-2004>.
- [23] "webbase-2001," <https://sparse.tamu.edu/LAW/webbase-2001>.
- [24] "web-google," <https://snap.stanford.edu/data/web-Google.html/>.
- [25] "clue-web," <https://networkrepository.com/web-ClueWeb09-50m.php>.
- [26] "bitcoin," <http://konect.cc/networks/soc-sign-bitcoinotc/>.
- [27] "soc-pokec," <https://snap.stanford.edu/data/soc-Pokec.html>.
- [28] "epinions," <http://konect.cc/networks/soc-Epinions1/>.
- [29] "math-overflow," <http://konect.cc/networks/sx-mathoverflow/>.
- [30] "amazon0505," <https://snap.stanford.edu/data/amazon0505.html>.
- [31] "wiki-talk," <https://snap.stanford.edu/data/wiki-Talk.html>.
- [32] "email-auall," <https://snap.stanford.edu/data/email-EuAll.html>.
- [33] "as-caida," <https://snap.stanford.edu/data/as-Caida.html>.
- [34] "wiki-vote," <https://snap.stanford.edu/data/wiki-Vote.html>.
- [35] "ArnetMiner Citation Network," <https://lfs.aminer.cn/lab-datasets/soinf/>.
- [36] J. Tang, J. Zhang, L. Yao, J. Li, L. Zhang, and Z. Su, "Arnetminer: extraction and mining of academic social networks," in *KDD*. ACM, 2008, pp. 990–998.
- [37] J. Tang, J. Sun, C. Wang, and Z. Yang, "Social influence analysis in large-scale networks," in *KDD*. ACM, 2009, pp. 807–816.
- [38] Q. Cheng, D. Yan, T. Wu, L. Yuan, J. Cheng, Z. Huang, and Y. Zhou, "Efficient enumeration of large maximal k -plexes," in *EDBT*. OpenProceedings.org, 2025, pp. 53–65.
- [39] A. Conte, D. Firmani, C. Mordente, M. Patrignani, and R. Torlone, "Fast enumeration of large k -plexes," in *KDD*. ACM, 2017, pp. 115–124.
- [40] J. Zheng, M. Jin, Y. Jin, and K. He, "Relaxed graph color bound for the maximum k -plex problem," *CoRR*, vol. abs/2301.07300, 2023.
- [41] L. Chang and K. Yao, "Maximum k -plex computation: Theory and practice," *Proc. ACM Manag. Data*, vol. 2, no. 1, pp. 63:1–63:26, 2024.
- [42] A. Ahmad, D. Yan, X. Chen, L. Yuan, Q. Zhang, and S. Adhikari, "Maximum k -plex finding: Choices of pruning techniques matter!" *Proc. VLDB Endow.*, vol. 18, no. 9, pp. 2928–2940, 2025.
- [43] C. Giatsidis, D. M. Thilikos, and M. Vazirgiannis, "D-cores: Measuring collaboration of directed graphs based on degeneracy," in *ICDM*. IEEE Computer Society, 2011, pp. 201–210.
- [44] Y. Fang, Z. Wang, R. Cheng, H. Wang, and J. Hu, "Effective and efficient community search over large directed graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 11, pp. 2093–2107, 2019.
- [45] C. Ma, Y. Fang, R. Cheng, L. V. S. Lakshmanan, W. Zhang, and X. Lin, "Efficient algorithms for densest subgraph discovery on large directed graphs," in *SIGMOD Conference*. ACM, 2020, pp. 1051–1066.
- [46] C. Ma, Y. Fang, R. Cheng, L. V. S. Lakshmanan, W. Zhang, and X. Lin, "On directed densest subgraph discovery," *ACM Trans. Database Syst.*, vol. 46, no. 4, pp. 13:1–13:45, 2021.
- [47] G. Liu and L. Wong, "Effective pruning techniques for mining quasi-cliques," in *ECML/PKDD*, ser. Lecture Notes in Computer Science, vol. 5212. Springer, 2008, pp. 33–49.
- [48] K. Yu, C. Long, D. P. and T. Chakraborty, "On efficient large maximal biplex discovery," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 824–829, 2023.
- [49] K. Yu, C. Long, S. Liu, and D. Yan, "Efficient algorithms for maximal k -biplex enumeration," in *SIGMOD*. ACM, 2022, pp. 860–873.
- [50] K. Yu and C. Long, "Maximum k -biplex search on bipartite graphs: A symmetric-bk branching approach," *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 49:1–49:26, 2023.

A Complexity Analysis.

Given a directed graph $G = (V, E)$, we evaluate the worst-case running time of Algorithm 4. Let $T(m, n)$ be the worst-case running time of function BK2(G, k_1, k_2, q, P, C, X) where $m = |C|$ and $n = |P \cup C|$, and we define $k = \max(k_1, k_2)$. Next, we analyze $T(m, n)$ by considering four different cases:

I. When $C = \emptyset$ (Lines 4–6), then

$$T(m, n) = T(0, n) \leq c_1, \quad (4)$$

where c_1 is a constant.

II. When $P \cup C$ is not a (k_1, k_2) -plex and the pivot vertex is in P (Lines 8–17). Here, $p + 1$ branches are created each with a smaller candidate set C' . Before each call the sets C' and X' are further shrunk in Lines 15–16 which takes time $O(|V|^2)$. The worst-case running time of this case is, therefore

$$T(m, n) \leq (p + 1)c_2|V|^2 + \sum_{i=1}^p T(m - i, n - 1) + T(m - d, n - d + p), \quad (5)$$

where c_2 is a constant.

III. When $P \cup C$ is a (k_1, k_2) -plex (Lines 19–24), then no further branches are created, and the cost in this case is only incurred by updating X' in Line 22, which is

$$T(m, n) \leq c_2|V|^2 \quad (6)$$

IV. When $P \cup C$ is not a (k_1, k_2) -plex and the pivot vertex is in C (Lines 25–30). The cost in this case is

$$T(m, n) \leq c_2|V|^2 + T(m - 1, n) + T(m - 1, n - 1).$$

In the worst case, no candidate vertex is reduced from $C - \{v_p\}$ in Line 28 so v_p remains the minimum degree vertex in the subsequent recursive call at Line 30 (within which $v_p \in P' = P \cup \{v_p\}$). Hence, by expanding $T(m - 1, n)$ using Equation (5) we have:

$$T(m, n) \leq (p + 2)c_2|V|^2 + \sum_{i=1}^p T(m - 1 - i, n - 1) + T(m - 1 - d, n - d + p) + T(m - 1, n - 1). \quad (7)$$

Next, we compute a closed-form upper-bound of $T(m, n)$ which satisfies the Equations (4)–(7), and we can drop the argument n from $T(m, n)$ since the base case expressions in Equations (4) and (6) are not a function of n . Then, the largest $T(m)$ is upper-bounded by Equations (4) and (7), i.e.

$$T(m) \leq \begin{cases} c_1 & \text{if } m = 0 \\ (p + 2)c_2|V|^2 + \sum_{i=0}^p T(m - 1 - i) & \\ +T(m - 1 - d) & \text{otherwise} \end{cases} \quad (8)$$

Since $p \leq d - 1$ and $p < k$, the worst-case is reached when $p = k - 1$ and $d = k$. Then, Equation (8) can be rewritten as:

$$T(m) \leq \begin{cases} c_1 & \text{if } m = 0 \\ (k + 1)c_2|V|^2 + \sum_{i=0}^k T(m - 1 - i) & \text{otherwise} \end{cases} \quad (9)$$

We show that $T(m) = O(\alpha_k \gamma_k^m - \beta_k)$ where γ_k is largest real root of $x^{k+2} - 2x^{k+1} + 1 = 0$, $\beta_k = \frac{(k+1)c_2|V|^2}{k}$ and $\alpha_k = \frac{(2^k k + 1)\beta_k + 2^k c_1}{\gamma_k^{k+1}}$.

LEMMA A.1. Let $H(m) = \alpha_k \gamma_k^m - \beta_k$ where γ_k is largest real root of $x^{k+2} - 2x^{k+1} + 1 = 0$, $\beta_k = \frac{(k+1)c_2|V|^2}{k}$ and $\alpha_k = \frac{(2^k k + 1)\beta_k + 2^k c_1}{\gamma_k^{k+1}}$. If $T(m)$ satisfies Equation (9), then $T(m) \leq H(m)$ for $m \geq 0$.

PROOF. We first show $T(m) \leq 2^{m-1}k\beta_k + 2^{m-1}c_1$ by induction on m .

Basis: for $m = 1$, $T(1) \leq (k + 1)c_2|V|^2 + T(0) \leq k\beta_k + c_1$.

Induction: We assume that $T(i) \leq 2^{i-1}k\beta_k + 2^{i-1}c_1$ for all $i = 1, \dots, m - 1$ and prove that it also holds for $i = m$. By Equation (9) we have:

$$\begin{aligned} T(m) &\leq (k + 1)c_2|V|^2 + T(m - 1) + \dots + T(0) \\ &= (k + 1)c_2|V|^2 + c_1 + \sum_{i=1}^{m-1} T(i) \\ &\leq (k + 1)c_2|V|^2 + c_1 + \sum_{i=1}^{m-1} (2^{i-1}k\beta_k + 2^{i-1}c_1) \quad (10) \\ &= k\beta_k(1 + \sum_{i=1}^{m-1} 2^{i-1}) + c_1(\sum_{i=1}^{m-1} 2^{i-1} + 1) \\ &= 2^{m-1}k\beta_k + 2^{m-1}c_1 \end{aligned}$$

Then, we prove that $T(m) \leq H(m)$ for all $m \geq 0$ again by induction on m .

Basis: for $m = 1$, $T(1) \leq (k + 1)c_2|V|^2 + T(0) \leq k\beta_k + c_1$, and $H(1) = \alpha_k \gamma_k - \beta_k = \frac{(2^k k + 1)\beta_k + 2^k c_1}{\gamma_k^k} \geq k\beta_k + c_1$ (since $\gamma_k < 2$ and hence $\gamma_k^k < 2^k$).

Induction: We assume that $T(m) \leq H(m)$ for $m = 1, \dots, i$ and prove that it also holds for $m = i + 1$. First, let us factorize $x^{k+1} - 2x^{k+1} + 1$ as $(x - 1)(x^{k+1} - x^k - \dots - 1)$. Denote $F_k(x) = (x^{k+1} - x^k - \dots - 1)$. We have $F_k(1) < 0$ and $F_k(2) > 0$, indicating that $F_k(x) = 0$ has a root between $(1, 2)$. Combining the fact that γ_k is also the root of $F_k(x) = 0$, thus $\gamma_k^k + \dots + 1 = \gamma_k^{k+1}$. Multiply both side with γ_k^{i-k} , we have $\gamma_k^i + \dots + \gamma_k^{i-k} = \gamma_k^{i+1}$. Then the following inequalities hold.

$$\begin{aligned} T(i + 1) &\leq T(i) + \dots + T(i - k) + (k + 1)c_2|V|^2 \\ &\leq \alpha_k(\gamma_k^i + \dots + \gamma_k^{i-k}) - (k + 1)\beta_k + k\beta_k \\ &\leq \alpha_k(\gamma_k^i + \dots + \gamma_k^{i-k}) - \beta_k \quad (11) \\ &= \alpha_k \gamma_k^{i+1} - \beta_k \\ &= H(i + 1) \quad \square \end{aligned}$$

Finally, as we call BK2(.) with $m = |V|$ and k is a constant, we get the worst-case running time of our algorithm as given in Theorem 6.1.

B Complete Results in Parallel Executions

We implemented the parallel version of Ours based on the description in Section 8. Table 5 reports the (k_1, k_2) -plex enumeration time of parallel Ours on all our datasets in Table 1 when running with 2, 4, 8, 16 and 32 threads, respectively, where we use the default

Table 5: The Running Time of Mining Large Maximal (k_1, k_2) -Plexes in Parallel Execution

Dataset	k_1	k_2	q	$\#(k_1, k_2)$ - plexes	Serial	2 Threads	4 Threads	8 Threads	16 Threads	32 Threads
bitcoin	2	5	10	144.00	0.06	0.04	0.02	0.01	0.01	0.02
			12	0	0.01	0.00	0.00	0.00	0.00	0.00
	3	5	10	24,261	8.35	4.78	2.46	1.29	0.66	0.36
			12	286	1.05	0.62	0.33	0.17	0.09	0.05
wiki-vote	3	5	10	474	1.85	1.08	0.55	0.29	0.15	0.08
			12	0	0.08	0.05	0.02	0.01	0.01	0.01
	4	5	10	30,222	349.32	197.94	102.45	52.57	26.57	16.44
			12	94	16.63	9.76	4.99	2.61	1.32	0.91
mathoverflow	2	2	10	523,633	28.55	18.48	9.48	4.93	2.50	1.34
			12	150,888	13.35	9.01	4.56	2.38	1.22	0.69
	2	3	10	1,762,917	63.16	47.81	24.41	12.71	6.45	3.52
			12	602,862	32.44	24.92	12.61	6.63	3.36	1.86
as-caida	2	3	10	23,314	7.99	0.34	0.19	0.10	0.08	0.15
			15	185	0.93	0.02	0.01	0.01	0.01	0.02
	3	4	15	17,303	14.87	0.63	0.33	0.18	0.10	0.10
			20	0	0.19	0.01	0.00	0.00	0.00	0.00
epinions	2	3	10	773,095	23.85	18.34	9.48	4.94	2.56	1.41
			12	252,511	10.33	8.06	4.18	2.23	1.19	0.80
	3	4	12	24,599,029	2,053.04	1,174.05	605.69	314.25	158.78	85.62
			15	2,382,084	381.99	229.27	118.32	61.53	31.62	18.13
email-euall	2	3	10	17,438	1.69	1.26	0.65	0.34	0.18	0.19
			12	1442	0.58	0.39	0.21	0.11	0.07	0.12
	3	4	12	196,878	80.58	46.44	23.49	12.18	6.14	3.38
			15	1351	9.19	5.20	2.68	1.40	0.72	0.44
amazon0505	3	5	10	4,696	0.27	0.08	0.04	0.02	0.02	0.01
			12	24	0.14	0.01	0.00	0.00	0.00	0.00
	5	3	10	1,532	0.20	0.04	0.02	0.01	0.01	0.00
			12	1	0.13	0.00	0.00	0.00	0.00	0.00
web-google	2	3	15	997	0.29	0.05	0.02	0.01	0.01	0.00
			20	57	0.21	0.01	0.01	0.00	0.00	0.00
	3	4	15	1,993	0.32	0.06	0.03	0.02	0.01	0.01
			20	61	0.22	0.02	0.01	0.00	0.00	0.00
soc-pokec	2	3	15	17,980	2.11	0.92	0.49	0.25	0.21	0.39
			20	17	0.95	0.03	0.01	0.01	0.01	0.02
	3	3	15	508,884	54.91	32.24	16.52	8.65	4.41	2.41
			20	648	1.66	0.49	0.25	0.13	0.07	0.06
wiki-talk	2	2	15	22,219	287.28	173.50	87.73	44.75	22.50	11.48
			20	0	69.70	38.84	19.89	10.27	5.17	2.67
	2	3	15	165,156	525.13	376.12	191.71	97.82	48.99	25.01
			20	0	129.02	77.30	39.23	20.22	10.16	5.21
arabic-2005	2	2	1000	106,895	3,906.70	1,897.35	1,017.31	527.40	269.04	140.58
	3	3	3000	3,500	111.60	60.20	29.56	15.10	8.69	4.64
uk-2005	2	2	200	117,255	301.09	162.70	84.11	43.34	21.82	11.38
	3	3	400	66,639	168.98	91.51	46.34	23.95	12.11	6.34
it-2004	2	2	1000	15,087	1,170.05	621.80	321.12	166.53	85.40	52.04
	3	3	3000	1,034	77.90	41.35	21.72	10.21	6.40	3.94
webbase-2001	2	2	300	327,882	646.41	320.84	164.46	90.02	53.56	30.75
	3	3	500	88,777	184.44	95.16	52.07	30	18.15	8.75
clue-web	2	2	10	1,215,977	26.05	14.15	7.66	3.82	1.75	1.69
	3	3	20	95,383	52.92	31.76	16.46	8.56	4.42	2.37

timeout threshold $\tau_{time} = 0.1$ ms is adopted, which is tested to work consistently well on various datasets.

As Table 5 shows, our parallel algorithm is efficient and scales up well with the number of CPU cores on all the datasets.